Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

**Ю.Е. Гапанюк**

# СЕМИНАР №1
# АНАЛИЗ РЕЗУЛЬТАТОВ КВАЛИФИКАЦИИ ГОНОЧНЫХ БОЛИДОВ

***Учебные материалы по дисциплине***
***«Архитектура автоматизированных систем обработки информации и управления»***
***Профиль: «Архитектура программных систем»***

Москва – 2026

# Оглавление


1 ЦЕЛЬ ..... 3

2 ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ ..... 3

3 ЭТАП 1: ВВОД ДАННЫХ ..... 3

4 ЭТАП 2: СТАТИСТИЧЕСКИЙ АНАЛИЗ ..... 4

5 ЭТАП 3: ВИЗУАЛИЗАЦИЯ В ВИДЕ ФОРМАТИРОВАННОЙ ТАБЛИЦЫ ..... 4

6 ЭТАП 4: СОРТИРОВКА (ТУРНИРНАЯ ТАБЛИЦА) ..... 5

7 ЭТАП 5: ФИЛЬТР ПО МИНИМАЛЬНОЙ СКОРОСТИ ..... 6

8 ЭТАП 6: РЕВИЗИЯ КОДА С ПОМОЩЬЮ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ ..... 6

9 ТЕСТОВЫЕ ДАННЫЕ (10 УЧАСТНИКОВ) ..... 7

10 ПРИМЕР ПРОГРАММНОЙ РЕАЛИЗАЦИИ ..... 9

# 1 Цель

Написать консольное приложение для анализа средних скоростей гоночных автомобилей на квалификационном заезде.

# 2 Требования к реализации

Используйте массивы, циклы for, преобразование типов и форматированный вывод.

# 3 Этап 1: Ввод данных

1. Спросите у пользователя количество участников заезда (например, `n`).
2. Создайте два массива длиной `n`:

   - string[] teams — для хранения названий команд/болидов.

   - double[] avgSpeeds — для хранения средней скорости на круге (км/ч).
3. 3. В цикле запросите ввод названия команды и средней скорости для каждого участника.

Подсказка: Используйте `double.Parse()` или `double.TryParse()` или `Convert...` для ввода скоростей.

Пример ввода данных:

```
Введите количество участников: 3

Участник #1
Команда: Red Bull Racing
Средняя скорость (км/ч): 228.5

Участник #2
Команда: Ferrari
Средняя скорость (км/ч): 226.3
```

```
Участник #3
Команда: Mercedes
Средняя скорость (км/ч): 224.8
```

## 4 Этап 2: Статистический анализ

После ввода данных программа должна рассчитать и вывести:

1. Среднюю скорость по всем участникам (сумма скоростей / количество). Вывод с точностью до 2 знаков после запятой: $"{avg:F2}".
2. Лидер квалификации: найти максимальную скорость и вывести название команды.
3. Самый медленный болид: найти минимальную скорость и вывести название команды.
4. Разница темпа: вычислить разницу между максимальной и минимальной скоростью.

Пример вывода данных:

```
--- СТАТИСТИКА КВАЛИФИКАЦИИ ---
Средняя скорость: 226.53 км/ч
Лидер: Red Bull Racing (228.50 км/ч)
Самый медленный: Mercedes (224.80 км/ч)
Разница темпа: 3.70 км/ч
```

## 5 Этап 3: Визуализация в виде форматированной таблицы

Выведите протокол квалификации в виде таблицы.

Пример вывода:

```
---------------------------------------------
| Команда              | Скорость (км/ч)     |
----------------------------------------------
| Red Bull Racing      |              228.50 |
| Ferrari              |              226.30 |
| Mercedes             |              224.80 |
---------------------------------------------
```

Подсказка:

- Для названий: $"{team,-20}" (выравнивание влево, ширина 20).
- Для скоростей: $"{speed,20:F2}" (выравнивание вправо, ширина 20, 2 знака).

## 6 Этап 4: Сортировка (Турнирная таблица)

Отсортируйте участников по убыванию скорости (от самого быстрого к самому медленному) — как в реальном протоколе квалификации.

1. Используйте произвольный (но реализованный Вами) алгоритм сортировки, укажите какой алгоритм Вы использовали.
2. При перестановке скоростей в массиве `avgSpeeds` обязательно переставляйте названия команд в массиве `teams`.

Пример вывода:

```
ИТОГОВЫЙ ПРОТОКОЛ КВАЛИФИКАЦИИ
-----------------------------------------------
| Поз. | Команда                | Скорость      |
-----------------------------------------------
|  1   | Red Bull Racing        |      228.50   |
|  2   | Ferrari                |      226.30   |
|  3   | Mercedes               |      224.80   |
-----------------------------------------------
```

## 7 Этап 5: Фильтр по минимальной скорости

Реализуйте поиск команд, чья скорость превышает заданный порог (например, для определения участников топ-10).

Пример:

```
Введите минимальную скорость для отбора (км/ч): 226

Команды со скоростью >= 226.00 км/ч:
- Red Bull Racing (228.50 км/ч)
- Ferrari (226.30 км/ч)

Отобрано команд: 2
```

## 8 Этап 6: Ревизия кода с помощью большой языковой модели

- - Проведите ревизию кода с помощью большой языковой модели.
- - Попросите большую языковую модель указать на слабые места в Вашем коде, оптимизировать код, использовать конструкции из

современных версий языка C# и при этом генерировать детальные комментарии по каждому исправлению.

- - Проверьте работоспособность сгенерированного кода.
- - Сравните сгенерированный код с написанным Вами кодом. Сколько изменений предложила большая языковая модель? Согласны ли Вы с ними? Действительно ли по Вашему мнению предложенные изменения улучшили код?

## 9 Тестовые данные (10 участников)

Используйте эти данные для проверки работоспособности программы:

```
10 (количество команд)

Red Bull Racing
228.5

Ferrari
226.3

Mercedes
224.8

McLaren
223.7

Alpine
221.4
```

Aston Martin

220.9

AlphaTauri

219.6

Alfa Romeo

218.2

Haas

216.5

Williams

215.1

Ожидаемые результаты:

Статистика:

- Средняя скорость: **221.50 км/ч**
- Лидер: **Red Bull Racing (228.50 км/ч)**
- Самый медленный: **Williams (215.10 км/ч)**
- Разница темпа: **13.40 км/ч**

После сортировки (топ-3):

1. Red Bull Racing — 228.50 км/ч
2. Ferrari — 226.30 км/ч
3. Mercedes — 224.80 км/ч

# 10 Пример программной реализации

```
using System;

// ================= ОСНОВНОЙ КОД ПРОГРАММЫ =================

Console.WriteLine("=== АНАЛИЗ КВАЛИФИКАЦИИ ГРАН-ПРИ ===");
Console.WriteLine();

// Ввод количества участников
Console.Write("Введите количество участников: ");
int n = int.Parse(Console.ReadLine());
Console.WriteLine();

// Создание массивов
string[] teams = new string[n];
double[] avgSpeeds = new double[n];

// Ввод данных
InputData(teams, avgSpeeds, n);

// Вычисление статистики
Console.WriteLine("--- СТАТИСТИКА КВАЛИФИКАЦИИ ---");
CalculateStatistics(teams, avgSpeeds, n);
Console.WriteLine();

// Вывод исходного порядка
Console.WriteLine("--- ИСХОДНЫЙ ПОРЯДОК ---");
PrintTable(teams, avgSpeeds, n, false);
Console.WriteLine();

// Копирование массивов для сортировки
string[] sortedTeams = new string[n];
double[] sortedSpeeds = new double[n];
CopyArrays(teams, avgSpeeds, sortedTeams, sortedSpeeds, n);

// Сортировка
BubbleSort(sortedTeams, sortedSpeeds, n);

// Вывод отсортированного протокола
Console.WriteLine("--- ИТОГОВЫЙ ПРОТОКОЛ КВАЛИФИКАЦИИ ---");
PrintTable(sortedTeams, sortedSpeeds, n, true);
Console.WriteLine();

// Дополнительно: фильтр по скорости
FilterBySpeed(sortedTeams, sortedSpeeds, n);

Console.WriteLine();
Console.Write("Нажмите любую клавишу для выхода...");
Console.ReadKey();

// ================= ФУНКЦИИ =================

/* Ввод данных о командах и скоростях */
static void InputData(string[] teams, double[] speeds, int n)
{
    for (int i = 0; i < n; i++)
    {
        Console.WriteLine($"Участник #{i + 1}");
        Console.Write("Команда: ");
        teams[i] = Console.ReadLine();

        Console.Write("Средняя скорость (км/ч): ");
```

```
        speeds[i] = double.Parse(Console.ReadLine());

        Console.WriteLine();
    }
}

/* Вычисление и вывод статистики */
static void CalculateStatistics(string[] teams, double[] speeds, int n)
{
    // Вычисление средней скорости
    double sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += speeds[i];
    }
    double average = sum / n;

    // Поиск максимальной и минимальной скорости
    double maxSpeed = speeds[0];
    double minSpeed = speeds[0];
    string fastestTeam = teams[0];
    string slowestTeam = teams[0];

    for (int i = 1; i < n; i++)
    {
        if (speeds[i] > maxSpeed)
        {
            maxSpeed = speeds[i];
            fastestTeam = teams[i];
        }

        if (speeds[i] < minSpeed)
        {
            minSpeed = speeds[i];
            slowestTeam = teams[i];
        }
    }

    // Вывод статистики
    Console.WriteLine($"Средняя скорость: {average:F2} км/ч");
    Console.WriteLine($"Лидер: {fastestTeam} ({maxSpeed:F2} км/ч)");
    Console.WriteLine($"Самый медленный: {slowestTeam} ({minSpeed:F2} км/ч)");
    Console.WriteLine($"Разница темпа: {maxSpeed - minSpeed:F2} км/ч");
}

/* Вывод таблицы результатов */
static void PrintTable(string[] teams, double[] speeds, int n, bool showPosition)
{
    Console.WriteLine("-----------------------------------------------");

    if (showPosition)
    {
        Console.WriteLine("| Поз. | Команда              | Скорость      |");
    }
    else
    {
        Console.WriteLine("| Команда              | Скорость (км/ч)     |");
    }

    Console.WriteLine("-----------------------------------------------");

    for (int i = 0; i < n; i++)
    {
        if (showPosition)
```

```
        {
            Console.WriteLine($"| {i + 1,4} | {teams[i],-20} | {speeds[i],13:F2}
|");
        }
        else
        {
            Console.WriteLine($"| {teams[i],-20} | {speeds[i],19:F2} |");
        }
    }

    Console.WriteLine("----------------------------------------------");
}

/* Пузырьковая сортировка по убыванию скорости */
static void BubbleSort(string[] teams, double[] speeds, int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            // Для сортировки по убыванию используем <
            if (speeds[j] < speeds[j + 1])
            {
                // Меняем скорости местами
                double tempSpeed = speeds[j];
                speeds[j] = speeds[j + 1];
                speeds[j + 1] = tempSpeed;

                // Меняем команды местами
                string tempTeam = teams[j];
                teams[j] = teams[j + 1];
                teams[j + 1] = tempTeam;
            }
        }
    }
}

/* Копирование массивов */
static void CopyArrays(string[] srcTeams, double[] srcSpeeds,
                       string[] dstTeams, double[] dstSpeeds, int n)
{
    for (int i = 0; i < n; i++)
    {
        dstTeams[i] = srcTeams[i];
        dstSpeeds[i] = srcSpeeds[i];
    }
}

/* Фильтрация по минимальной скорости */
static void FilterBySpeed(string[] teams, double[] speeds, int n)
{
    Console.WriteLine("--- ДОПОЛНИТЕЛЬНО: ФИЛЬТР ПО СКОРОСТИ ---");
    Console.Write("Введите минимальную скорость для отбора (км/ч): ");
    double minSpeed = double.Parse(Console.ReadLine());
    Console.WriteLine();

    Console.WriteLine($"Команды со скоростью >= {minSpeed:F2} км/ч:");

    int count = 0;
    for (int i = 0; i < n; i++)
    {
        if (speeds[i] >= minSpeed)
        {
            Console.WriteLine($"- {teams[i]} ({speeds[i]:F2} км/ч)");
```

```
            count++;
        }
    }

    Console.WriteLine();
    Console.WriteLine($"Отобрано команд: {count}");
}
```